PIAZ LANG

A Domain-Specific Language & Laboratory Environment
for Advanced Networking, Systems, Binary, Crypto & Authorized Security Engineering

Internal Technical Proposal — Expanded Edition v1.2
Prepared for Company Red Team & Infrastructure Engineering

Perspective: 30-year IT & Network Engineering Practitioner
Version 1.2 | 2026

Packets • Flows • Tunnels • Custom Protocols • P2P Mesh • VPN
Signaling • FFT • HMM / Markov • Graph Engine • ZKP
DNS Algorithms • eBPF • Memory & Hex • Assembly • Bit Ops
Hashing & Custom Crypto • JIT / AOT • Channels & Concurrency

1. Executive Summary

Piaz Lang is a purpose-built language and laboratory whose core vocabulary covers networks, flows, packets, tunnels, P2P mesh, VPN, signaling, spectral analysis (FFT), probabilistic state models (Markov / HMM), zero-knowledge proofs, DNS algorithms, kernel hooks, memory views, graphs, and cryptographic building blocks. The design goal is speed from idea to running, observable, audited artifact while every sensitive capability remains under explicit authorization.

2. Design Principles (selected)

- Network-native & protocol-extensible — endpoints, flows, tunnels, custom wire protocols.
- P2P mesh & VPN — first-class mesh overlays and VPN setup primitives.
- Signaling & spectral — built-in signaling patterns and FFT for signal/protocol analysis.
- Probabilistic models — Markov chains and Hidden Markov Models as language constructs.
- ZKP-native — zero-knowledge proof circuits and verification as core services.
- DNS-complete — full suite of DNS algorithms and record manipulation.
- Low-level accessible — memory, hex, assembly, bit ops, OS APIs (gated).
- Concurrent & compiled — channels, atomics, JIT + AOT.
- Observable & auditable — telemetry and authorization context by default.

3. Core Networking, Tunnels & Custom Protocols (recap)

```
:8080 => 10.0.0.5:80 via ssh://user@jump.example
path = ssh://u1@b1 -> ssh://u2@b2 -> ssh://u3@target
protocol ShadowTalk { handshake = {...}; frame = length-prefixed(u32)+payload; ... }
sock = listen :9900 protocol ShadowTalk
```

4. Extended Capabilities with Syntax Examples

4.1 P2P Mesh & Built-in VPN Setup

First-class peer-to-peer mesh overlays and VPN configuration. Nodes discover, form encrypted meshes, and expose virtual interfaces. Useful for distributed labs and authorized multi-host exercises.

```
// Start a mesh node
mesh = p2p.mesh.start(name: "lab-mesh", listen: ":7700",
crypto: "x25519+chacha20poly1305")
mesh.join("seed1.example:7700")
mesh.join("seed2.example:7700")

// VPN overlay on top of the mesh
vpn = vpn.create(name: "lab-vpn", backend: mesh,
addr: "10.99.0.3/24", dns: ["10.99.0.1"])
vpn.up()
vpn.route_add("10.10.0.0/16", via: "10.99.0.1")

// Peer RPC still works over mesh
peer = mesh.peer("sensor-7")
flows = peer.call("collect_flows", "eth0", 5)
```

4.2 Built-in Signaling

Signaling primitives for session setup, offer/answer, ICE-like candidate exchange, and custom control-plane messages — usable for media, data channels, or lab orchestration.

```
sig = signaling.channel(mesh, topic: "session-42")

// Offer / answer pattern
offer = signaling.create_offer(caps: ["data", "audio"])
sig.send(offer)
answer = sig.recv(timeout: 5s)
session = signaling.accept(answer)

// Custom control messages
sig.emit("start_capture", {iface: "eth0", filter: "tcp"})
sig.on("start_capture") { cfg => capture(cfg.iface, cfg.filter) }
```

4.3 Built-in FFT (Fast Fourier Transform)

FFT / IFFT and related spectral tools for signal analysis, covert-channel research, protocol fingerprinting, and any workload that benefits from frequency-domain views.

```
samples = capture_audio(rate: 48000, secs: 1) // or any f32/f64 vector
spec = fft.forward(samples)
power = fft.power_spectrum(spec)
peaks = fft.find_peaks(power, threshold: 0.15)

filtered = fft.bandpass(spec, low: 300, high: 3400)
time_dom = fft.inverse(filtered)

// Sliding / windowed
```

```
stft = fft.stft(samples, window: "hann", hop: 256)
```

4.4 Built-in Markov Chains & Hidden Markov Models

First-class definition of Markov chains and Hidden Markov Models for protocol state inference, anomaly detection, traffic modeling, and sequence prediction inside the lab.

```
// Markov chain
mc = markov.chain {
states: ["Idle", "SynSent", "Established", "FinWait"]
transitions: {
Idle -> SynSent : 0.7
Idle -> Idle : 0.3
SynSent -> Established : 0.9
Established -> FinWait : 0.2
Established -> Established : 0.8
}
}
next = mc.sample("Established")
path = mc.generate(start: "Idle", steps: 20)

// Hidden Markov Model
hmm = markov.hmm {
hidden: ["Browse", "Exfil", "Idle"]
observed: ["dns", "https", "large_upload", "silence"]
start: { Browse: 0.5, Idle: 0.4, Exfil: 0.1 }
emit: {
Browse: { dns: 0.4, https: 0.5, silence: 0.1 }
Exfil: { large_upload: 0.7, https: 0.2, dns: 0.1 }
}
// transition matrix omitted for brevity
}
states = hmm.viterbi(observations)
prob = hmm.forward(observations)
```

4.5 Built-in Zero-Knowledge Proofs (ZKP)

Circuit definition, proof generation, and verification as language services. Intended for privacy-preserving lab protocols, selective disclosure, and research into verifiable computation.

```
// Define a simple circuit (e.g. range + preimage)
circuit = zkp.circuit {
public age_commit: Field
private age: u8, nonce: Field
constrain {
age >= 18
age_commit == hash(age, nonce)
}
}

proof = zkp.prove(circuit, { age: 21, nonce: n, age_commit: c })
ok = zkp.verify(circuit, proof, { age_commit: c })

// Attach to a P2P / protocol handshake
protocol PrivateAge {
handshake = {
send zkp.prove(...)
recv peer_proof
assert zkp.verify(circuit, peer_proof, public_inputs)
}
}
```

4.6 Built-in DNS Algorithms & Operations

Full suite of DNS record types, resolution strategies, zone manipulation, DNSSEC helpers, and common algorithms (round-robin, weighted, latency-based, etc.) as native operations.

```
// Resolution
answers = dns.resolve("api.internal", type: "A", server: "10.0.0.53")
answers = dns.resolve("svc.ns1", type: "SRV")

// Zone / record construction
zone = dns.zone("lab.example")
zone.add(dns.A("app", "10.10.1.5", ttl: 30))
zone.add(dns.AAAA("app", "fd00::5"))
zone.add(dns.SRV("_sip._tcp", priority: 10, weight: 20, port: 5060, target: "sip.lab.example"))
zone.add(dns.TXT("_verify", "ok"))
zone.sign(dnssec_key) // DNSSEC helper

// Algorithms / policies
pool = dns.pool(["10.1.0.1", "10.1.0.2", "10.1.0.3"])
next = pool.round_robin()
next = pool.weighted({ "10.1.0.1": 3, "10.1.0.2": 1 })
next = pool.latency_based(probe: true)
```

```
// Exfil / research helpers (authorized lab only)
dns.exfil_encode(data, domain: "tunnel.lab.example", mode: "txt")
data = dns.exfil_decode(captured_queries)
```

4.7 P2P Instance Calls & Code Import (recap)

```
import "lab/scanners.piaz" as scan
export fn collect_flows(iface: str, secs: int) -> FlowSet { ... }
flows = p2p://sensor-1/collect_flows("eth0", 5)
```

4.8 Channels, Threads, Atomics

```
ch = channel<Packet>(capacity: 1024)
go { for p in capture("eth0") { ch.send(p) } }
counter = atomic<u64>(0); counter.fetch_add(1)
mu.lock() { shared_map.insert(k, v) }
```

4.9 JIT / AOT

```
piaz run lab_scenario.piaz
piaz build --aot --opt 2 lab_scenario.piaz -o scenario
#[jit] fn fast_filter(p: Packet) -> bool { ... }
```

4.10 Memory, OS APIs, Hex, Assembly, Bit Ops

```
mem = memory.dump(addr: 0x7fabc000, len: 4096)
view = memory.view(pid: lab_pid, addr: 0x400000, len: 0x1000, mode: "r")
hx = hex.view(bin); hits = hx.find("48 89 e5"); hx.patch(offset: 0x2a30, bytes: "90 90 90 90")
stub = asm.assemble("mov rax, 1\nret", arch: "x86_64")
flags.set_bit(7, true); hdr = bitview(packet[0..4]); version = hdr[0..4]
```

4.11 Codecs / Data Hiding

```
codec = codec.define { encode(msg) { interleave(msg, noise) |> xor_keystream(key) } ... }
wire = codec.encode(real_payload)
```

4.12 Graph & State-Machine Engine (incl. classic algorithms)

```
sm = state_machine { states: [Idle, Ready, Closing]; transitions: { Idle --syn--> Ready; ... } }
g = graph(); g.add_node(...); g.edge(...); g.run(...)
path = g.shortest_path("A", "Z", weight: "latency")
tour = g.tsp_approx("start", cities); flow = g.max_flow("s", "t")
```

4.13 Hashing, Keypairs, HMAC, Custom Encryption Protocols

```
h = hash.blake3(data); kp = crypto.keypair.generate("x25519")
tag = crypto.hmac(key, data, hash: "sha256")
protocol SecureLab { handshake = { ephemeral = keypair.generate("x25519"); ... } }
conn = dial("peer:9900", protocol: SecureLab)
```

5. High-Level Architecture (updated)

Layer	Responsibility
Surface Language	Syntax, modules, P2P/mesh/VPN, signaling, protocol & circuit DSLs, Markov/HMM, graph DSL
Compiler + IR	Parse, type-check, JIT specialization, AOT native, optional eBPF lowering
Runtime	Scheduler, channels, atomics, tunnels, mesh fabric, signaling, telemetry, audit
Signal & Model Services	FFT/STFT, Markov chains, HMM (Viterbi/forward), graph algorithms
Crypto & ZKP	Hashes, keypairs, HMAC, AEAD, KDF, ZKP prove/verify, protocol composition
DNS Service	Resolution, zone construction, DNSSEC helpers, load-balancing algorithms
Systems / Memory	Raw sockets, OS APIs (gated), memory views, hex, assembly helpers
Lab Orchestrator	Namespaces, topologies, scenario runners, artifact management
Audit & Policy	Authorization context, mandatory logging, capability gating

6. Security, Ethics & Compliance Posture

Mesh, VPN, memory views, binary mutation, ZKP, DNS helpers, and custom protocols are dual-use. Design bias remains strongly toward authorized, auditable operation: explicit authorization context, structured audit for every sensitive action, lab-scoped binary/memory operations, and internal-first distribution of the most powerful capabilities.

7. Suggested Roadmap (updated)

Phase	Focus	Outcomes
0 — Foundation	Core language, runtime, channels, sockets, flows	Runnable concurrent networking scripts
1 — Tunnels + Mesh + VPN	Forwarding, P2P mesh, VPN overlays, signaling	Replace ad-hoc tunnel/VPN glue
2 — Protocols + DNS + Packets	Custom protocols, DNS suite, packet model, codecs	New wire + name-service logic in-language
3 — Signal + Probabilistic	FFT/STFT, Markov chains, HMM	Spectral and sequence modeling in lab
4 — Low-level + Graph + ZKP + Crypto	Memory/hex/asm, graph engine, ZKP, hashing/keypairs	Full lab surface in one environment
5 — JIT/AOT + Hardening	Production AOT, policy engine, audit completeness	Internal production-ready platform

8. Closing Recommendation

Approve a Phase 0/1 spike focused on core language, channels, tunnels, P2P mesh, and basic VPN/signaling. Success metric: existing scripts and manual procedures retired in favor of short, readable, audited Piaz programs. Later phases layer DNS, FFT/HMM, ZKP, graph, and low-level services onto a stable foundation.

Document status: Internal proposal — not for external distribution.
Piaz Lang concept, syntax sketches, and capability list are company confidential.